

Dependent Multiplicities in Dependent Linear Type Theory

Maximilian Doré

maximilian.dore@cs.ox.ac.uk

Abstract

We present a novel dependent linear type theory in which the multiplicity of some variable—i.e., the number of times the variable can be used in a program—can depend on other variables. This allows us to give precise resource annotations to programs involving branching and recursion that cannot be adequately typed in other theories. Our type system is obtained by embedding linear logic into dependent type theory and specifying how the embedded logic interacts with the host theory. We can then use the natural numbers of the dependent type theory to derive a quantitative typing system with dependent multiplicities. Our theory supports W-types, thereby giving a principled resource-aware treatment of a large class of inductive types. We characterise the semantics as Categories with Families indexed in symmetric monoidal categories, thereby generalising Quantitative Categories with Families. Existing dependently typed languages can easily be extended with our system, which we demonstrate with an implementation in Agda.

CCS Concepts

• **Theory of computation** → **Linear logic; Type theory; Logic and verification; Program specifications; Type structures.**

Keywords

Dependent Type Theory, Linear Logic, Quantitative Type Theory

ACM Reference Format:

Maximilian Doré. . Dependent Multiplicities in Dependent Linear Type Theory. In *Proceedings of* . ACM, New York, NY, USA, 13 pages.

1 Introduction

Girard’s linear logic [29] has seen exciting and diverse applications in computer science, ranging from concurrency [19] over quantum programming languages [4] to complexity theory [11, 21]. When we view propositions as types, linear logic gives rise to a programming language whose type system ensures that any program uses exactly the resources it received. A natural extension of linear logic is to not only allow single usage of a variable, but to instead equip each variable with a *multiplicity* which specifies how often that variable is to be used. For example, a copying function will have multiplicity 2 for the input variable in such quantitative type systems. Linear Haskell [16], Quantitative Type Theory (QTT) [9, 39] and Graded Modal Type Theory [2, 44] have applied this idea fruitfully to give practical programming languages in which the type of a program specifies how all inputs are being used, giving guidance to both the programmer who wants to be confident in the correctness of a program and to the compiler which attempts to find efficient

ways to execute it. However, the typing disciplines of those systems quickly becomes restrictive when we use data types that do not represent resources but instead implement program logic and organise the actual resources (which might, e.g., be values on the heap, file handlers, messages passed along some channel, etc. [18, 47]). Consider the booleans, which has the following elimination principle in such a linear or quantitative system.

$$\frac{b : \mathbb{B} \quad \Delta \Vdash x : A \quad \Delta \Vdash y : A}{\Delta \Vdash \text{if } b \text{ then } x \text{ else } y : A}$$

Some boolean b , which does not require any resources, is used to decide which of the two elements of type A we return. Both elements of type A have to be constructed from the same resources, which is a steep ask: different if-then-else branches in a program will generally do different things, and hence use different resources.

The limitations of current resource-sensitive type theories get more present if we introduce more features to our programming language. In a higher-order function, how some parameter is used might depend on the values of other parameters, making it impossible to precisely type many functions. Similarly, inductive types introduce a dynamic usage of resource that cannot be captured. Consider an inductive type of binary trees, $\text{BTree } A$, which stores values of type A in the leaves. Suppose we want to map a function f onto such trees, i.e., a program which would have the following type in Agda [43].

$$(t : \text{BTree } A) \rightarrow (f : A \rightarrow B) \rightarrow \text{BTree } B$$

An implementation of this type should take a binary tree t and invoke f for any leaf of the t . However, no existing linear or quantitative system allows us to express this use of f . Again, the issue is that the usage of some resource *depends* on some value that is only known at runtime.

Did somebody say a type depends on some value? Enter Martin-Löf’s dependent type theory (DTT) [38]. DTT equips intuitionistic logic with predicates, which allows for expressing detailed specifications in the type of a program [37]. It seems natural to expect that a combination of DTT with linear logic should be able to deal with the phenomena described above, and the promise of combining both systems has long excited researchers [20, 21, 35, 51]. However, work on dependent linear type theory was long plagued by an apparent conundrum: in DTT, a type can depend on a term, does this constitute a use of the term? The answer most researchers gave was an unemphatic “yes” and disallowed dependency on linear variables, until McBride [39] argued that the answer should be “no” since types are just there for contemplation, and not for computation. In his QTT, further worked out by Atkey [9], variables are equipped with multiplicities drawn from some resource algebra and the structural rules of DTT are restricted to take these multiplicities into account. While QTT provides a full combination of linearity and dependency, it still has a crucial shortcoming: multiplicities are



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

static elements of the resource algebra, thereby making it impossible for some multiplicity to depend on some other variables or to capture the runtime dynamics of a recursive program.

In this paper we show how to take the approach of McBride further: we should contemplate terms not only in types, but also in multiplicities. By combining DTT with linear logic in a quite natural fashion, we obtain a highly expressive type system in which we can capture all of the above described phenomena.

Approach. Our theory comes with two entailment relations $\vdash$ and $\Vdash$, where the former incarnates a standard intuitionistic DTT, called the *host theory*, while the latter is a resource-aware calculus (our setup can be thought of akin to the 0- and 1-fragments of QTT). We cannot hypothesise over new variables in $\Vdash$, but only repeat values that were constructed in the $\vdash$ fragment, consequently, we call the hypotheses of $\Vdash$ *supplies* (and not “contexts”). These supplies are subject to the standard structural rules of linear logic. Our setup can be considered a deep embedding of linear logic into DTT. In particular, supplies form a type in the host theory that we can eliminate into, which allows us compute for a given natural number $\Gamma \vdash m : \mathbb{N}$ the m -fold copy of a supply Δ , denoted $\Delta^{\wedge m}$. This operation allows us to equip variables with multiplicities, giving rise to a quantitative type system. Since m can depend on variables in Γ , our notion of multiplicity is dynamic, allowing us to express that a multiplicity *depends* on some other variables.

We present linear versions of the usual type formers of DTT, such as functions $(x : A)^{\wedge m} \multimap B(x)$ whose inputs can be used m times (we will omit $\wedge m$ in case m is 1), and booleans $\mathbb{B}^{\circ}$, whose eliminator allows for different resources to be used in each branch (we write $|-|$ for a function which embeds $\mathbb{B}^{\circ}$ into $\mathbb{N}$).

$$\frac{b : \mathbb{B}^{\circ} \quad \Delta_0 \Vdash x : A \quad \Delta_1 \Vdash y : A}{\Delta_0^{\wedge |b|}, \Delta_1^{\wedge |-b|} \Vdash \text{if } b \text{ then } x \text{ else } y : A}$$

Moreover, we can give precise types to many higher-order functions, in particular those involving inductive types. We can implement a map function for binary trees whose type precisely captures how often the mapped function is used, where leafs t computes the number of leaves in t .

$$(t : \text{BTree } A) \multimap (f : A \multimap B)^{\wedge (\text{leafs } t)} \multimap \text{BTree } B$$

Semantics for our language can be characterised as a rather straightforward combination of models of DTT, namely Categories with Families [27], with models of linear logic, namely symmetric monoidal categories; we call the resulting structure *linear Categories with Families*. Our notion of model of dependent linear type theory has been foreshadowed by Vákár [51], the main contribution of our semantics lies not in the novelty of the given structure, but rather that it is enough to interpret our highly expressive syntax. In particular, we show how our semantics justifies intricate introduction and elimination rules for linear W-types, thereby giving a principled answer to how inductive types can be treated in a linear system. Our notion generalises certain Quantitative Categories with Families, which have been proposed as the semantics for QTT [9], further substantiating our claim that the presented theory is a more expressive version of QTT. In particular, we can mimic the realisability model given by Atkey [9], which justifies erasing the $\vdash$ fragment and only keeping terms derived in $\Vdash$ for computation.

Since we do not change the structural rules of our host theory, we can relatively easily add our system to existing dependently typed languages. We demonstrate this with an implementation in Agda, using the semantic interpretation of the linear judgments and type formers as a guide for a highly rigorous (if not particularly practical) type-checker.

Summary.

- We present syntax for a novel *dependent linear type theory* (Section 2) in which variables have *dependent multiplicities*, allowing us to precisely type programs that cannot be typed in other linear or quantitative systems. Our language has function types, pair types and standard algebraic data types, more specifically W-types whose positions are finite. We give a syntactic characterisation of a class of *clearly linear* programs which can all be precisely typed and are hence valid programs in our linear typing discipline (Theorem 2.1).
- Models of our syntax can be characterised as a rather direct combination of standards model of DTT and linear logic that we call *linear Categories with Families* (Section 3); we will show how to interpret all type formers (Lemmas 3.1–3.4); evince a realisability model which demonstrates that the underlying DTT can be erased for computation (Example 3.2); and show how our model generalises Quantitative Categories with Families [9] (Lemma 3.5).
- We show in Section 4 that the full power of intuitionistic logic can be recovered in $\Vdash$ in standard fashion with a linear exponential comonad “!” to annotate variables which can be used in unrestricted fashion (Lemma 4.1).
- Since our theory is a combination of a standard DTT with an embedded linear logic, we can add our type system to existing languages, which we demonstrate with an implementation of our theory in Agda (Section 5).

We will close with a discussion of related (Section 6) and future work (Section 7). The accompanying artefact¹ contains the implementation of our type system in Agda with all examples, and provides a formalisation of Lemmas 3.1–3.4.

2 Syntax for Dependent Linear Type Theory

Our type system is based on intuitionistic type theory in the style of Martin-Löf [38], which we call the *host theory* or *intuitionistic theory* (the linear logic we will embed is also non-classical and hence “intuitionistic”, but we will use this adjective only for the host theory). We have the standard judgements $\Gamma \text{ ctxt}$ for Γ being a *context*; $\Gamma \vdash A \text{ ityp}$ for A being an *intuitionistic type* in context Γ ; and $\Gamma \vdash a : A$ for a being an *intuitionistic term* of type A in context Γ . We assume that we have a cumulative hierarchy of universes, for which also simply write *ityp*, omitting universe levels (which can be consistently assigned, as demonstrated in the artefact). Our theory has explicit simultaneous substitutions, where we write $\Gamma' \vdash \gamma : \Gamma$ for a substitution γ which turns types and terms in context Γ into ones living in Γ' . The substitution calculus is standard and we refer to Angiuli and Gratzer [8] for a modern presentation. We write $\Gamma \vdash a \equiv a' : A$ if two terms $a, a' : A$ are definitionally equal and $\text{Id}_A(a, a')$ for a propositional identity type which we assume to be

¹<https://github.com/anonforlics26/dltt>

intensional. Such an equality is present in, e.g., the type theories underlying Agda [43] and RoqC [13], but also in homotopy type theory [50] and cubical type theory [17].

We distinguish a collection of ground types, denoted A gtyp, any one of which is also an intuitionistic type. Our theory comes with standard data types and type formers where we write $\star : \top$ for the single inhabitant of the unit type; $\perp$ for the type with no inhabitants; $\text{true}, \text{false} : \mathbb{B}$ for the booleans; $(x, y) : (x : A) \times B(x)$ for the dependent pair type, which we define positively with pair constructor $(-, -)$ and an eliminator $\text{let } (x, y) = - \text{ in } -$; and $\lambda x.b : (x : A) \rightarrow B(x)$ for the dependent function type. Moreover, our host theory comes with W -types [38], written $\mathcal{W}(x : A, B(x))$, where we call A the *constructors* and $B(x)$ the *positions*. When eliminating a given $p : \mathcal{W}(x : A, B(x))$ into some dependent motive C using $\text{elim}_{\mathcal{W}}(\text{afg}, r, p)$, r can use the given constructor a , subtree $f : B(a) \rightarrow \mathcal{W}(x : A, B(x))$ and recursively computed value $g : (y : B(a)) \rightarrow C(f y)$. In case we are eliminating into non-dependent types, we do not need the f and will simply write $\text{elim}_{\mathcal{W}}(\text{ag}, r, p)$.

We write 0 and succ for the constructors of the natural numbers $\mathbb{N}$, defined as the standard W -type, and $\text{elim}_{\mathbb{N}}(s, z, m)$ for the eliminator of $m : \mathbb{N}$ with inductive case s and base case z . The map $|-| : \mathbb{B} \rightarrow \mathbb{N}$ sends false to 0 and true to 1 .

2.1 Linear Judgements and Structural Rules

We now embed linear logic into our intuitionistic host theory by adding the following judgements.

- $\Gamma \vdash \Delta \text{ sply}$ asserts that Δ is a *supply* in context Γ .
- $\Gamma \vdash A \text{ ltyp}$ asserts that A is a *linear type* in context Γ .
- $\Delta \Vdash a : A$ asserts that a is a *linear term* of linear type A derivable from Δ , where $\Gamma \vdash A \text{ ltyp}$ and $\Gamma \vdash \Delta \text{ sply}$.

As a first intuition, supplies can be thought of as “linear contexts”, and we will see that the standard structural rules of linear logic apply to supplies. However, supplies will not introduce new variables, but rather arbitrary terms. They are hence used to specify which resources are to be used in a construction, but they themselves live in some intuitionistic context. Similarly, our linear types live in some intuitionistic context. We embed linear supplies and types into our host theory by stipulating that these form in fact *intuitionistic types*. This will allow us to compute with resources annotations in our host theory.

$$\frac{}{\Gamma \vdash \text{sply ityp}} \text{splyEMB} \quad \frac{}{\Gamma \vdash \text{ltyp ityp}} \text{ltypEMB}$$

Note that as intuitionistic types, sply and ltyp are quite unusual since we will not stipulate any elimination rules. Instead, we find it helpful to think of the intuitionistic host theory as giving the universe of discourse for our linear calculus.

Our distinction between intuitionistic types and linear types should not be taken akin to the dichotomy introduced by, e.g., Cervesato and Pfenning [20], Vákár [51] or Krishnaswami et al. [35]; rather, linear types should be thought of as substructural, more fine-grained versions of the intuitionistic types. Throughout this section will define by structural induction an operation $(-)^{\bullet}$ on the linear type formers which turns any linear type into an

intuitionistic type, giving rise to the following admissible rule.

$$\frac{\Gamma \vdash A \text{ ltyp}}{\Gamma \vdash A^{\bullet} \text{ ityp}}$$

Similarly, we will also be able to regard any linear term as an intuitionistic term, but we will do this translation implicitly. Whenever a term appears in a type, it is an (implicitly translated) intuitionistic term, e.g., we might consider $B(a)$ where a might have originally been a linear term. Akin to the approach of McBride [39] and Atkey [9], we use the host theory to model dependencies between linear types—i.e., a linear type B depending on some other linear type A is represented by the judgement $x : A^{\bullet} \vdash B(x) \text{ ltyp}$.

The first linear types that we introduce are the ground types. The values of ground types are those that we actually care about as resources, while all the type formers introduced in Section 2.2 only allow us to rearrange and combine such values.

$$\frac{\Gamma \vdash A \text{ gtyp}}{\Gamma \vdash A \text{ ltyp}} \text{!gtyp}$$

For ground types A we define $A^{\bullet} := A$.

Structural rules. The rules governing supplies are standard for a linear calculus. The following introduce the empty supply and the join of two supplies living in the same intuitionistic context.

$$\frac{}{\Gamma \vdash \diamond \text{ sply}} \diamond \text{sply} \quad \frac{\Gamma \vdash \Delta_0 \text{ sply} \quad \Gamma \vdash \Delta_1 \text{ sply}}{\Gamma \vdash \Delta_0, \Delta_1 \text{ sply}}, \text{sply}$$

Additionally, we can create supplies which hypothesise over a variable of a linear type, which lives in a context extended with a variable of the underlying intuitionistic type. Using this supply we can then stipulate the *variable* rule of linear logic, sometimes also called the identity or axiom rule.

$$\frac{\Gamma \vdash A \text{ ltyp}}{\Gamma, x : A^{\bullet} \vdash x : A \text{ sply}} \text{VARsply} \quad \frac{\Gamma \vdash A \text{ ltyp}}{x : A \Vdash x : A} \text{VAR}^{\circ}$$

Our host theory therefore takes care of introducing new variables, which our supplies may incorporate.

Lastly, we introduce the central structural rule for linear logic which allows us to reorder the assumptions of a supply.

$$\frac{\Delta_0, \Delta_1 \Vdash \mathcal{J}}{\Delta_1, \Delta_0 \Vdash \mathcal{J}} \text{EXCH}$$

We furthermore stipulate structural rules which allow us to remove $\diamond$ from non-empty contexts and treat “,” as an associative operation: we have rules that make $\Delta, \diamond \Vdash \mathcal{J}$ and $\Delta \Vdash \mathcal{J}$ interderivable; as well as $(\Delta_0, \Delta_1), \Delta_2 \Vdash \mathcal{J}$ and $\Delta_0, (\Delta_1, \Delta_2) \Vdash \mathcal{J}$.

Substitutions. Our host theory has taken care of most of the non-linear structural part of our theory, such as modelling type dependencies, and we will also just reuse the intuitionistic substitutions for the $\Vdash$ fragment of our theory. In particular, we can substitute a linear term for a variable using its intuitionistic translation.

The central substitution rule for $\Vdash$ establishes that when applying a substitution, we have to apply it to the supply, linear type and linear term at the same time.

$$\frac{\Gamma' \vdash \gamma : \Gamma \quad \Delta \Vdash a : A}{\Delta[\gamma] \Vdash a[\gamma] : A[\gamma]} \text{ltmSb}$$

We have omitted rules which state that $\Delta[\gamma]$ and $A[\gamma]$ also live in Γ' , and that substitutions factor through supplies, e.g., $(\Delta_0, \Delta_1)[\gamma] \equiv \Delta_0[\gamma], \Delta_1[\gamma]$, until they are at the level of variables where substitution is defined as usual for intuitionistic variables.

Our substitution rule allows us to put arbitrary terms into supplies, e.g., we can derive for any term $a : A$ that $a : A \Vdash a : A$ by substituting a for x in the variable judgement $x : A \Vdash x : A$. Supplies are hence not “contexts”, since they can contain arbitrary terms. Note that substitutions might in general duplicate or drop resources—but since we apply a substitution to both the assumption and conclusion of a linear judgement, we maintain that $\Vdash$ captures linear derivability. In particular, we can apply the weakening substitution of the host theory to a linear judgement, which means $x : A$ is derivable from supply $x : A$ in context $x : A^\bullet, y : B(x)^\bullet$ —crucially, we have only weakened the context in which our linear derivation lives, but not the resources given by our supply. Also note that we can reorder terms which ever way we like in a supply, e.g., $y : B(x), x : A$ is a supply living in the aforementioned context. Since our host theory takes care of variable dependencies, we can unperturbedly apply the structural rules of linear logic.

We stipulate the usual type conversion rule for linear types, i.e., if two linear types A and A' are equal and $\Delta \Vdash a : A$, then $\Delta \Vdash a : A'$. We also add a rule which allows us to use a *proof* that two supplies are equal to coerce a linear term judgement.

$$\frac{\Delta \Vdash \mathcal{J} \quad \Gamma \vdash \text{Id}_{\text{sply}}(\Delta, \Delta')}{\Delta' \Vdash \mathcal{J}} \text{ splyConv}$$

Computing with supplies. So far, we have introduced an intuitionistic type theory with an embedded linear type system without mentioning multiplicities in the spirit of QTT. It turns out that we do not have to stipulate any more rules to obtain a quantitative calculus—since our supplies are embedded as intuitionistic types, we can eliminate into supplies, and thereby compute with these. It is not only convenient that our structural rules do not have to take into account multiplicities, we will see that this way of deriving multiplicities is significantly more expressive since it allows us to specify dynamic resource annotations.

A very useful type to compute supplies with are the natural numbers, and these will serve as our main notion of multiplicity.

DEFINITION 2.1. Let $\Delta \wedge m := \text{elim}_{\mathbb{N}}((-, \Delta), \diamond, m)$ for $\Gamma \vdash \Delta \text{ sply}$ and $\Gamma \vdash m : \mathbb{N}$.

Note that the multiplicity m can mention other variables in the context. The exact number of Δ 's generated by $\Delta \wedge m$ can therefore differ at runtime, depending on how the variables in the context are initialised.

Relation with quantitative type theory. We often talk about our language as an extension to QTT [9, 39], for readers familiar with QTT it might be helpful to make the relation between both theories explicit (we will study the relation between our theory and QTT semantically in Section 3.3).

The term judgement of QTT is $\Gamma \vdash a :^\sigma A$, where σ is either 0 or 1—in our setting, the 0-fragment corresponds to $\vdash$, while the 1-fragment corresponds to $\Vdash$.

In QTT, a context Γ is of the form $x_1 :^{p_1} A_1, \dots, x_n :^{p_n} A_n$, which corresponds to the supply $(x_1 : A_1)^\wedge p_1, \dots, (x_n : A_n)^\wedge p_n$, living

in context $x_1 : A_1^\bullet, \dots, x_n : A_n^\bullet$. The 0-ing operation of QTT hence just amounts to considering the context underlying a supply.

QTT is parametrised over the resource algebra that is used for the multiplicities, in case of the natural numbers we can recover the structural rules of QTT in our theory, where we only make use of closed, i.e., static natural numbers m in $\Delta \wedge m$. Context addition of QTT is simply taking the join of two supplies, while context scaling is derived in our setting in Definition 2.1. The equations for these operations (which hold definitionally in QTT) can be shown propositionally by induction on the natural numbers.

The variable rule of QTT is reflected in our setting by weakening Var° appropriately often.

2.2 Type Formers with Dynamic Multiplicities

We will now introduce linear type formers, which are akin to the usual intuitionistic type formers, but annotated with resource specifications, making use of Definition 2.1 rules to hypothesise over, e.g., m copies of some supply for some open term of the natural numbers. We have already stipulated with !gtyp that any ground type is a linear type, and in fact, only values of ground types are considered resources in our system. All type formers introduced in this section will just serve to program with and organise resources, but do not constitute resources themselves.

Function types. We will denote the linear analogue of the dependent function type as usual with a “ $\multimap$ ”. The rules of this type are similar to those of QTT [9], with the crucial difference that multiplicities are now terms of the natural numbers built from the same context as the linear types, instead of static elements of some resource algebra.

$$\frac{\Gamma \vdash A \text{ ltyp} \quad \Gamma, x : A^\bullet \vdash B(x) \text{ ltyp} \quad \Gamma \vdash m : \mathbb{N}}{\Gamma \vdash (x : A)^\wedge m \multimap B(x) \text{ ltyp}} \multimap F$$

$$\frac{\Gamma \vdash \Delta \text{ sply} \quad \Delta, (x : A)^\wedge m \Vdash b : B(x)}{\Delta \Vdash \lambda x^\wedge m. b : (x : A)^\wedge m \multimap B(x)} \multimap I$$

$$\frac{\Delta_0 \Vdash f : (x : A)^\wedge m \multimap B(x) \quad \Delta_1 \Vdash a : A}{\Delta_0, \Delta_1 \wedge m \Vdash f a : B(a)} \multimap \text{APP}$$

$$\frac{\Delta_0, (x : A)^\wedge m \Vdash b(x) : B(x) \quad \Delta_1 \Vdash a : A}{\Delta_0, \Delta_1 \wedge m \Vdash (\lambda x^\wedge m. b) a \equiv b[a/x] : B(a)} \multimap \beta$$

In the introduction rule, the supply $\Delta, (x : A)^\wedge m$ lives in context $\Gamma, x : A^\bullet$, while in the conclusion Δ lives just in Γ . When introducing a function, we hence abstract over the variable in both the intuitionistic context and the linear supply.

Note that the “ $\wedge m$ ” in the linear function type is part of the syntax, while exponentiation in the supplies is just using Definition 2.1 (we will see in the semantics in Section 3.2 that both notions of exponentiation coincide however).

We will not spell out how substitutions act on our linear type and term formers as this works analogously to their intuitionistic counterparts, only taking into account also the multiplicity parameters. For example, substitutions pass through the function type in the expected way.

$$((x : A)^\wedge m \multimap B(x))[\gamma] \equiv (x : A[\gamma])^\wedge m[\gamma] \multimap B(x)[\gamma]$$

The intuitionistic type underlying a linear function type is given as $((x : A) \wedge m \multimap B(x))^\bullet := (x : A^\bullet) \rightarrow B(x)^\bullet$. We will also implicitly translate functions $\lambda x \wedge m. b$ to their intuitionistic counterparts $\lambda x. b$. We will omit any multiplicities in types if they are 1.

Pair types. In a similar vein, our linear pairs equip the first argument with a multiplicity, which again is a potentially open term of the natural numbers.

$$\frac{\Gamma \vdash A \text{ ltyp} \quad \Gamma, x : A^\bullet \vdash B(x) \text{ ltyp} \quad \Gamma \vdash m : \mathbb{N}}{\Gamma \vdash (x : A) \wedge m \otimes B(x) \text{ ltyp}} \otimes F$$

$$\frac{\Delta_0 \Vdash a : A \quad \Delta_1 \Vdash b : B(a)}{\Delta_0 \wedge m, \Delta_1 \Vdash (a \wedge m, b) : (x : A) \wedge m \otimes B(x)} \otimes I$$

Define $((x : A) \wedge m \otimes B(x))^\bullet := (x : A^\bullet) \times B(x)^\bullet$, and let us implicitly translate a linear term $(x \wedge m, y)$ to an intuitionistic term (x, y) where necessary. Eliminating into some linear type C , which depends on the pair type and hence lives in $\Gamma, z : (x : A^\bullet) \times B(x)^\bullet$, works similarly to QTT, with both Δ_0 and Δ_1 living in Γ .

$$\frac{\Delta_0 \Vdash p : (x : A) \wedge m \otimes B(x) \quad \Delta_1, (x : A) \wedge m, y : B(x) \Vdash c : C(x, y)}{\Delta_0, \Delta_1 \Vdash \text{let } (x, y) = p \text{ in } c : C(p)} \otimes E$$

Going from the last premise to the conclusion, we abstract over the variables x and y in both the context and supply. The pair type is subject to the usual substitution [8] and computation rules [12].

Recall that our host theory has standard dependent pairs, so we can define projections for intuitionistic pairs p .

$$\text{fst } p := \text{let } (x, y) = p \text{ in } x \quad \text{snd } p := \text{let } (x, y) = p \text{ in } y$$

We hence have a substitution from context $x : A^\bullet$ to context $z : (x : A^\bullet) \times B(x)^\bullet$ which replaces a variable x with $\text{fst } z$. Applying this to the variable judgement $x : A \Vdash x : A$ allows us to derive the linear judgement $\text{fst } z : A \Vdash \text{fst } z : A$. Note that while this linear judgement lives in context $z : (x : A^\bullet) \times B(x)^\bullet$, all it shows is that the first projection of z is enough to derive itself. Prima facie, it might seem that we endanger our linear typing discipline by applying intuitionistic substitutions to our linear terms, but this turns out to be fine since we also subject our supplies to these substitutions, thereby maintaining a calculus which neither drops nor duplicates values.

Empty, unit and booleans. With our theory we want to capture *value linearity*, which means that we do not consider elements of data types such as the unit or booleans as resources. It would also be sensible to consider a unit type as resource-relevant akin to the diamond type of Hofmann [32] and Atkey [10] to reason about the runtime of programs; or to consider booleans a resource to capture reversible computations [3]. While these are exciting lines of work that we hope to explore in the future, we will in the following focus on value linearity as in QTT [9, 39], ensuring that any program written in $\Vdash$ uses exactly the provided values of ground types.

The unit type works exactly like in QTT. Constructing the unique element of the unit type does not require any resources, and we need to explicitly eliminate elements of the unit type, which will

require the joint resources of the element we are eliminating and the motive C which depends on the unit type.

$$\frac{}{\diamond \vdash \star : \top^\circ} \top^\circ I \quad \frac{\Delta_0 \Vdash a : \top^\circ \quad \Delta_1 \Vdash c : C(\star)}{\Delta_0, \Delta_1 \Vdash \text{let } \star = a \text{ in } c : C(a)} \top^\circ E$$

The computation and substitution rules follow again in standard fashion and we define $(\top^\circ)^\bullet := \top$.

In order to devise interesting inductive types as W-types we will also need a linear version of the empty type, which can be introduced in any context, does not have any constructors, and gives a linear version of the explosion principle for any linear type C depending on $\perp^\circ$.

$$\frac{\Delta \Vdash e : \perp^\circ}{\Delta \Vdash \text{elim}_{\perp^\circ}(e) : C(e)} \perp^\circ E$$

We define $(\perp^\circ)^\bullet := \perp$.

Our dynamic notion of multiplicity becomes very useful for booleans. While our introduction rules are similar to those of QTT—we assume that we do not require any resources to introduce a boolean—our elimination rule is considerably more general since the resources used can differ between the branches.

$$\frac{}{\diamond \vdash \text{true}, \text{false} : \mathbb{B}^\circ} \mathbb{B}^\circ I$$

$$\frac{\Gamma \vdash b : \mathbb{B} \quad \Delta_0 \Vdash c_0 : C(\text{true}) \quad \Delta_1 \Vdash c_1 : C(\text{false})}{\Delta_0 \wedge |b|, \Delta_1 \wedge |\neg b| \Vdash \text{if } b \text{ then } c_0 \text{ else } c_1 : C(b)} \mathbb{B}^\circ E$$

Computation and substitution rules follow again in standard fashion, and we define $(\mathbb{B}^\circ)^\bullet := \mathbb{B}$.

Our boolean eliminator allows us to precisely annotate the resources used by programs which branch, which constitutes a very useful refinement of linear logic. Defining the coproduct using dependent linear types, we also have available a dynamic version of another prominent connective of linear logic.

EXAMPLE 2.1. We define additive disjunction as

$$A \oplus B := (b : \mathbb{B}^\circ) \otimes (\text{if } b \text{ then } A \text{ else } B)$$

and its constructors as $\text{inl } x := (\text{true}, x)$ and $\text{inr } y := (\text{false}, y)$. Using the eliminator for linear booleans we can derive the following dynamic elimination into some linear type C depending on $A \oplus B$.

$$\frac{\Delta_0 \Vdash z : A \oplus B \quad \Delta_1, x : A \Vdash c_1 : C(\text{inl } x) \quad \Delta_2, y : B \Vdash c_2 : C(\text{inr } y)}{\Delta_0, \Delta_1 \wedge |\text{fst } z|, \Delta_2 \wedge |\neg \text{fst } z| \Vdash \text{case } z \text{ have } c_1 \text{ else } c_2 : C(z)} \Delta$$

Relevant for the development of inductive types will also be this application of the coproduct which uses elimination into linear types to obtain finite linear types.

EXAMPLE 2.2. We define $\text{Fin } n := \text{elim}_{\mathbb{N}}((-) \oplus \top^\circ, \perp^\circ, n)$. Note that we perform induction on an intuitionistic term n to obtain a *linear* type with n elements. We can show by induction on n that we do not need any resources to construct some $k : \text{Fin } n$.

In the following, we will write $B \text{ ltyp}_{\text{fin}}$ if B has been constructed as such a finite type $\text{Fin } n$. Δ

Inductive types. The ability to compute with supplies gives us a lot of expressivity, which we will now use to incorporate inductive types in our linear system. We use Martin-Löf's W-types [38] since they capture most inductive types used in functional programming languages and can naturally be extended to, e.g., inductive families and coinductive types. While we expect that our approach can be adapted for these classes of types, we leave this to future work.

We restrict our attention to W-types whose positions are finite. This is because our supplies are closed only under *finite* join “ $\cdot$ ”, and we will need to gather resources for all positions. Intuitively, having finite positions means that any constructor has only finitely many recursive fields, which is a relatively modest assumption. In particular, all algebraic data types can be expressed as such finitary W-types (e.g., also rose trees as the constructor type may be infinite).

The formation rule for our linear $\mathcal{W}^\circ$ type is parametrised with a dynamic multiplicity which allows us to specify how many copies of the values present in each constructor are to be used.

$$\frac{\Gamma \vdash A \text{ ltyp} \quad \Gamma, x : A^\bullet \vdash B(x) \text{ ltyp}_{\text{fin}} \quad \Gamma \vdash m : \mathbb{N}}{\Gamma \vdash \mathcal{W}^\circ(x : A^\bullet m, B(x)) \text{ ltyp}} \mathcal{W}^\circ F$$

For example, the multiplicity m can be used to type lists in which every list element can be used m -many times. We could have also made the m dependent on the given $x : A$ to allow varying multiplicities for different constructors, this is a rather straightforward generalisation which would however complicate our presentation.

Before we can formulate introduction and elimination rules, we need to extend our syntax with an operator $\bigotimes$ which represents the join of a supply depending on some finite type.

$$\frac{\Gamma \vdash B \text{ ltyp}_{\text{fin}} \quad \Gamma, y : B^\bullet \vdash \Delta \text{ sply}}{\Gamma \vdash \bigotimes_{y:B} \Delta(y) \text{ sply}} \text{finsply}$$

For closed n , this construct simply computes the join of all supplies depending on $\text{Fin } n$, i.e., $\bigotimes_{y:\text{Fin } n} \Delta(y)$ is definitionally equal to $\Delta(y_1), \dots, \Delta(y_n)$.

The introduction rule for linear W-types requires an element $a : A$, constructed using some resources Δ_0 , and for each position a prescription which element of the recursive data type it points to. These recursive elements might be made up of different resources, which we capture with the supply Δ_1 that depends on the given position. The resulting element of our $\mathcal{W}^\circ$ type consequently uses m copies of Δ_0 as well all the join of all Δ_1 's for each position.

$$\frac{\Delta_0 \Vdash a : A \quad \Gamma, y : B(a)^\bullet \vdash \Delta_1 \text{ sply} \quad \Delta_1 \Vdash f : \mathcal{W}^\circ(x : A^\bullet m, B(x))}{\Delta_0 \wedge m, \bigotimes_{y:B(a)} \Delta_1(y) \Vdash \text{sup}(a^\bullet m, f) : \mathcal{W}^\circ(x : A^\bullet m, B(x))} \mathcal{W}^\circ I$$

When eliminating an element p of $\mathcal{W}^\circ(A^\bullet m, B)$, we have to specify for any constructor $x : A$ how to construct an element of the motive C . This element can be constructed with m copies of x , a supply Δ_1 specific for this constructor, and the join of all recursively constructed elements. To define the eliminator into dependent motives, we assume that the context of the second premise contains a function $f : B(x)^\bullet \rightarrow \mathcal{W}(A^\bullet, B^\bullet)$ giving us the subtrees for all positions of a given constructor $x : A^\bullet$, and a function $g : B(x)^\bullet \rightarrow C(f y)^\bullet$ which computes the actual values.

$$\frac{\Delta_0 \Vdash p : \mathcal{W}^\circ(x : A^\bullet m, B(x)) \quad (x : A)^\bullet m, \Delta_1(x) \cdot \bigotimes_{y:B(a)} (g y : C(f y)) \Vdash c(x, f, g) : C(\text{sup}(x, f))}{\Delta_0, \text{elim}_{\mathcal{W}}(x \Delta, (\Delta_1(x) \cdot \bigotimes_{y:B(a)} \Delta(y)), p) \Vdash \text{elim}_{\mathcal{W}^\circ}(c, p) : C(p)} \mathcal{W}^\circ E$$

The eliminated element requires the supply that makes up p , as well as some resources recursively computed as follows: for the given constructor x , we need its associated resources $\Delta_1(x)$, as well as all the recursively computed resources Δ for each position.

In the computation rule, we are given a specific constructor a and a function f_a which gives us the subtree at a .

$$\frac{\Delta_0 \Vdash a : A \quad \Gamma, y : B(a)^\bullet \vdash f_a(y) : \mathcal{W}(A^\bullet, B^\bullet) \quad (x : A)^\bullet m, \Delta_1(x) \cdot \bigotimes_{y:B(a)} (g y : C(f y)) \Vdash c(x, f, g) : C(\text{sup}(x, f))}{\Delta_0 \wedge m, \Delta_1(a) \cdot \bigotimes_{y:B(a)} \text{elim}_{\mathcal{W}}(c, f_a(y)) : C(f_a(y)) \Vdash \text{elim}_{\mathcal{W}^\circ}(c, \text{sup}(a, f_a)) \equiv c(a, f_a, y. \text{elim}_{\mathcal{W}^\circ}(c, f_a(y))) : C(\text{sup}(a, f_a))} \mathcal{W}^\circ C$$

In the conclusion, we need to gather all resources associated with each subtree alongside m copies of the supply that made up a and a copy of the supply associated with this kind of constructor.

The substitution rules follow in the expected fashion, we define $(\mathcal{W}^\circ(x : A^\bullet m, B(x)))^\bullet := \mathcal{W}(x : A^\bullet, B(x)^\bullet)$.

The elimination rule for general $\mathcal{W}^\bullet$ types is very general, and instances of it are, e.g., the demolition principles for lists proposed by McBride [39]. It is quite a mouthful however, let us illustrate it with a simple example.

EXAMPLE 2.3. Defining binary trees as

$$\text{BTree } A := \mathcal{W}^\circ(c : \top^\circ \oplus A, \text{if fst } c \text{ then } \mathbb{B}^\circ \text{ else } \perp^\circ)$$

and implementing an intuitionistic function $\text{leafs} : \text{BTree } A^\bullet \rightarrow \mathbb{N}$ we can show by induction on the binary tree that the following recursor is admissible.

$$\frac{\Delta_0 \Vdash t : \text{BTree } A \quad y : C, z : C \Vdash n(y, z) : C \quad \Delta_1, x : A \Vdash l(x) : C}{\Delta_0, \Delta_1 \wedge (\text{leafs } t) \Vdash \text{rec}_{\text{BTree}}(n, l, t) : C}$$

To fold a binary tree t using a *node* function which is linear and a *leaf* function which takes in some resources Δ_1 as well as the value of the leaf, we need the resources that made up t and $(\text{leafs } t)$ -many copies of Δ_1 . Using this recursor we can easily implement the map function for binary trees discussed in Section 1.

We can generalise the recursor in different ways, and all of these generalisations follow from $\mathcal{W}^\circ E$: we can also have a supply for the recursive case, which will then be in the conclusion as many times as the number of nodes in the tree; we can use our multiplicity parameter for $\mathcal{W}^\circ$ -types to allow multiple copies of each leaf value to be used; and we can turn the recursor into a dependent eliminator. $\triangle$

2.3 Completeness and Type-Checking

Before turning to a semantical study of our system, we will in this section gather some results that we can derive on syntactic grounds.

Our translation from linear to intuitionistic types and terms allows us to regard any term derived in the linear fragment as one

in the intuitionistic fragment (akin to changing from the 1- to the 0-fragment in QTT [9, Lemma 2.2]).

LEMMA 2.1. The following rule is admissible.

$$\frac{\Gamma \vdash \Delta \text{ sply} \quad \Delta \Vdash a : A}{\Gamma \vdash a : A^\bullet}$$

As a corollary, we can conclude consistency of $\Vdash$ since the underlying dependent intuitionistic type theory is consistent [7].

Novel about our theory is that we can go further in the other direction than any other system—we can give precise resource annotations to many intuitionistic terms whose resources have to be approximated in any other system.

DEFINITION 2.2. The *clearly linear* terms are intuitionistic terms built without function introduction and pair and $\mathcal{W}$ elimination, i.e., all $\Gamma \vdash a : A$ such that a does not contain $\lambda(-).$, $(-).$, $\text{elim}_{\mathcal{W}}(-, -)$ and $\text{let } (x, y) = (-) \text{ in } (-)$. $\triangle$

To see why we excluded certain terms from our definition of clearly linear terms here some examples for intuitionistic terms which cannot have a linear counterpart.

EXAMPLE 2.4. Some intuitionistic higher-order functions cannot be represented as linear functions since the multiplicity dependencies between different variables cannot be ordered, e.g., if there are mutual dependencies. Consider a cartesian product function for lists, which takes in two lists xs and ys and returns a list of all pairs between these lists. We will need length xs copies of ys and length ys copies of xs , which makes it impossible to apply $\multimap$ I (note however that a cartesian product which is not internalised as a function in two arguments can be typed in $\Vdash$). $\triangle$

EXAMPLE 2.5. The intuitionistic projection fst is defined using an eliminator which discards the second element, which is not possible with our linear pairs (in contrast, we can derive a function $\text{snd} : (z : (A \multimap B)) \multimap B$ ($\text{fst } z$), which is desirable in a dependent system where we might only contemplate A). $\triangle$

EXAMPLE 2.6. We have no guarantee that an elimination principle for $\mathcal{W}$ -types is not dropping the recursively computed values, e.g., we might implement a constant function using $\text{elim}_{\mathcal{W}}(-, -)$. Such a program is clearly not linear. $\triangle$

With this syntactic restriction at hand, we can formulate the following completeness result for clearly linear terms. Our result is reminiscent of the completeness result of Dal Lago and Gaboardi [21], who proved for their dependent linear system that it can type all PCF programs. Crucially, their linear calculus is about the runtime of programs, whereas our system is about value linearity. As we argued with the above examples, not all programs maintain the given values, so we believe that the below result is as far as we can go with the level of intensionality considered here. Notably, our completeness result incorporates eliminators for booleans, i.e., we can type programs with arbitrary branching.

THEOREM 2.1 (COMPLETENESS OF $\Vdash$ FOR CLEARLY LINEAR TERMS). For any clearly linear term $\Gamma \vdash a : A$ there exist $\Gamma \vdash A^\circ \text{ ltyp}$ and $\Gamma \vdash \Delta \text{ sply}$ such that $(A^\circ)^\bullet = A$ and $\Delta \Vdash a : A^\circ$.

PROOF. By structural induction on the intuitionistic term a , where we strengthen the induction hypothesis to also prove that any term of $\top^\circ$ and $\mathbb{B}^\circ$ requires no resources. The cases are straightforward,

e.g., in case of the eliminator for booleans our induction hypothesis gives us different supplies for each branch, which we can combine in the conclusion of $\mathbb{B}^\circ \text{E}$. $\square$

The expressivity of our type system comes at a cost, however: since we can compute with supplies using our intuitionistic theory, type-checking for $\Vdash$ is vastly undecidable. For example, we might have to compare $\Delta \wedge m$ with $\Delta \wedge n$ for two different open terms of the natural numbers m and n . Dal Lago and Gaboardi [21] have dealt with this issue by parametrising their theory over the set function symbols that are allowed in the computation of resource annotations, restricting to simpler theories when type-checking needs to be decidable. We take a different approach and allow the full power of DTT for the computation of resource annotations since we can also utilise another feature of our theory: if we can establish that some terms are propositionally equal, we can use this witness in splyConv to convince the type-checker that we have used the specified resources. E.g., the following rule is admissible for any $\Gamma \vdash m, n : \mathbb{N}$.

$$\frac{\Delta_0 \wedge m, \Delta_1 \Vdash \mathcal{J} \quad \Gamma \vdash \text{Id}_{\mathbb{N}}(m, n)}{\Delta_0 \wedge n, \Delta_1 \Vdash \mathcal{J}}$$

3 Modelling Dependent Linear Type Theory

We will now characterise what constitutes a model of our syntax. Our semantics shares similarity with that of Vákár [51, 52], who modelled his dependent linear type theory using a category indexed in symmetric monoidal closed categories. Our model will be based on a similar indexed category, however, we can use a standard model of DTT, namely Categories with Families (CwF) [27], for our base category, while Vákár had to restrict context extension to reflect that types cannot depend on linear variables. In our system, we do not distinguish between intuitionistic (“cartesian” in the terminology of Vákár) and linear variables.

Our semantics has to reflect that we can compute in the host theory with supplies and linear types, we will hence need to model these as types in the CwF. We introduce a technique which achieves this in Section 3.1 and explain how to interpret our structural rules. Most type formers can be interpreted straightforwardly without requiring any more structure on our model as we will see in Section 3.2, with only function types requiring a bit more care since they interact with the main structural rule of the host theory, namely context extension (but again, the structure that we need here is not very surprising and has appeared in some variations in prior works [10, 51]).

We will give some concrete models in Section 3.3, namely a syntactic model and a realisability model. The latter closely resembles the model of Atkey [9] and underlines that we can erase the $\vdash$ fragment for computation. We have motivated our system as a generalisation of QTT, we give a semantic argument for this by showing that any model for our syntax is also a model of QTT which uses the natural numbers as resource algebra.

3.1 Embedding Linear Logic in Type Theory

Our syntax was based on two entailment relations, $\vdash$ for the intuitionistic host theory and $\Vdash$ for the embedded linear logic. Since the

$\vdash$ fragment is a completely standard DTT, we can use a standard model for this part of our language [27].

DEFINITION 3.1. A *Category with Families* (CwF) is given by a category Cx , whose objects are called *contexts* and morphisms *substitutions*, which has

- a terminal object, called the *empty context*,
- a presheaf of *types*, $\mathcal{T}y : \mathbf{Psh}(Cx)$, and a presheaf of *terms*, $\mathcal{T}m : \mathbf{Psh}(\int_{Cx} \mathcal{T}y)$, where for some $\gamma : \Gamma' \rightarrow \Gamma$ we write $_[\gamma]$ for the actions $\mathcal{T}y(\gamma) : \mathcal{T}y(\Gamma) \rightarrow \mathcal{T}y(\Gamma')$ and $\mathcal{T}m(\gamma, A) : \mathcal{T}m(\Gamma, A) \rightarrow \mathcal{T}m(\Gamma', A[\gamma])$,
- and *context extension*, i.e., for any $\Gamma : Cx$ and $A \in \mathcal{T}y(\Gamma)$ we have a context $\Gamma.A$, substitution $p_A : \Gamma.A \rightarrow \Gamma$ and term $q_A \in \mathcal{T}m(\Gamma.A, A[p_A])$ with the following universal property: for each $\gamma : \Gamma' \rightarrow \Gamma$ and $a \in \mathcal{T}m(\Gamma, A[\gamma])$ there is a substitution $\gamma.a : \Gamma' \rightarrow \Gamma.A$ such that $p \circ (\gamma.a) = \gamma$ and $q[\gamma.a] = a$ which is unique in the sense that for any $\gamma' : \Gamma' \rightarrow \Gamma.A$ we have $(p \circ \gamma').q[\gamma'] = \gamma$.

We distinguish a collection of ground types $\mathcal{T}y_0$, i.e., $\mathcal{T}y_0$ is a subpresheaf of $\mathcal{T}y$ stable under reindexing. We define $\mathcal{T}m_0$ to gather all terms of ground types at some context as

$$\mathcal{T}m_0(\Gamma) := \coprod_{A : \mathcal{T}y_0(\Gamma)} \mathcal{T}m(\Gamma, A).$$

$\mathcal{T}m_0$ is defined componentwise on substitutions. $\triangle$

We motivated our supplies in Section 2 as “linear contexts” consisting of values that can be derived in some intuitionistic context, correspondingly, we want to send each context Γ to a model of linear logic. Moreover, we want to be able to eliminate into the type of supplies, for which we embedded supplies as intuitionistic types. Let us capture this idea for general indexed categories.

DEFINITION 3.2. A CwF Cx *embeds* a functor $\mathcal{F} : Cx^{op} \rightarrow \mathbf{Cat}$ if for any $\Gamma : Cx$ we have an $F_\Gamma \in \mathcal{T}y(\Gamma)$ such that

$$\mathcal{T}m(\Gamma, F_\Gamma) \cong ob(\mathcal{F}(\Gamma))$$

and $hom : \mathcal{T}m(\Gamma, F_\Gamma) \rightarrow \mathcal{T}m(\Gamma, F_\Gamma) \rightarrow \mathcal{T}y(\Gamma)$ such that

$$\coprod_{x, y \in \mathcal{T}m(\Gamma, F_\Gamma)} \mathcal{T}m(\Gamma, hom\ x\ y) \cong \coprod_{x, y \in \mathcal{F}(\Gamma)} hom_{\mathcal{F}(\Gamma)}(x, y),$$

all natural in Γ . $\triangle$

Note that our embedded categories do not come with universal properties in the sense of elimination principles—we only want to construct supplies using the host theory, but never map out of a supply or its hom-set. We also assumed that linear types form an intuitionistic type, to model this we will use dependent pair types.

DEFINITION 3.3. A CwF Cx supports dependent pairs if for any $\Gamma : Cx$ we have an operation $(-) \times (-) : (\coprod_{A : \mathcal{T}y(\Gamma)} \mathcal{T}y(\Gamma, A)) \rightarrow \mathcal{T}y(\Gamma)$ and $\mathcal{T}m(\Gamma, A \times B) \cong \coprod_{a : \mathcal{T}m(\Gamma, A)} \mathcal{T}m(\Gamma, B[id.a])$ all natural in Γ . $\triangle$

We thus have everything at hand to model the structural rules of our theory given in Section 2.1.

DEFINITION 3.4. A *linear Category with Families* (lCwF) is a CwF Cx supporting pair types with an embedded functor $Sp : Cx^{op} \rightarrow \mathbf{SMCat}$ and a natural transformation $\iota : \mathcal{T}m_0 \rightarrow Sp$. $\triangle$

The $\vdash$ fragment of our syntax is interpreted as usual in the underlying CwF [31]. The linear judgements we introduced in Section 2.1 are interpreted as follows.

- $\Gamma \vdash \Delta$ *sply* is modelled as $\Delta : Sp(\Gamma)$.

- $\Gamma \vdash A$ *ltp* is modelled as a pair $\mathcal{T}y(\Gamma) \times Sp(\Gamma.A)$.

Given some $(A, \Delta_A) \in \mathcal{T}y(\Gamma) \times Sp(\Gamma.A)$ and $a \in \mathcal{T}m(\Gamma, A)$ we will write $\Delta_A(a)$ for $\Delta_A[id.a]$.

- $\Delta \Vdash a : A$, with linear type A modelled with (A, Δ_A) , is modelled as a pair $(a : \mathcal{T}m(\Gamma, A)) \times hom_{Sp(\Gamma)}(\Delta, \Delta_A(a))$.

The rules making supplies an intuitionistic type (*splyEMB*), the substitution rule for linear terms (*ltmSB*) and the conversion rule (*splyCONV*) are modelled by our embedding of supplies and their morphisms. The embedding of linear types (*ltpEMB*) is modelled by internalising the linear types using universes and the pair type of the host theory. Note that the translation $A^\bullet$, which turns any linear type into its underlying intuitionistic type, amounts to the first projection in our semantics. The linear variable rule (*Var^o*) is modelled using the variable of the underlying type theory $q_A \in \mathcal{T}m(\Gamma.A, A[p_A])$ with the identity morphism $id_{\Delta_A(q_A)}$ for a given linear type (A, Δ_A) . The inclusion of ground types as linear types (*lgtyp*) utilises the inclusion ι , i.e., A *ltp* as a linear type is (A, ι) .

Since the quantitative aspects of our core theory were derived and not stipulated (Definition 2.1), we are finished interpreting the structural rules. However, in order to interpret the type formers, we have to model supply exponentiation since this was explicit part of the syntax for linear function, pair and W-types. To prepare the ground for this, we note that $(-)^\wedge m$ can be considered a functor for any $\Gamma \vdash m : \mathbb{N}$, where the functorial action on $\delta : \Delta_0 \rightarrow \Delta_1$ is

$$\delta^\wedge m := \text{elim}_{\mathbb{N}}((_ \otimes \delta), id_\circ, m) : hom(\Delta_0^\wedge m)(\Delta_1^\wedge m).$$

In the following, we will interpret the type formers that came equipped with some multiplicity “ $^\wedge m$ ” instead as parametrised by some functor E , this flexibility will be useful in Section 4.

3.2 Interpreting the Linear Type Formers

We will now see how an lCwF incorporates the linear types we introduced in Section 2.2. We do not need much additional structure since we can utilise that our supplies live inside the host theory, more specifically, the dependent elimination principles of the intuitionistic types underlying a linear type former will do most—in the case of positive types all—of the work for us.

For the pair type, we have everything at hand since the CwF underlying an lCwF is assumed to support pairs. We interpret linear pair types equipped with some functor E , which was $(-)^\wedge m$ in our syntax in Section 2.

LEMMA 3.1. Any lCwF (Cx, Sp) models $E\otimes$.

PROOF. Given linear types (A, Δ_A) and (B, Δ_B) , we interpret $A \overset{E}{\otimes} B$ as $(A \times B, \text{let } (x, y) = q \text{ in } E(\Delta_A(x)) \otimes \Delta_B(y))$, where we utilise the embedding of supplies and write $\text{let } (x, y) = - \text{ in } -$ for the unique object given by the universal property of the intuitionistic pair type.

In the introduction rule, we have $a \in \mathcal{T}m(\Gamma, A)$, $\delta_0 : \Delta_0 \Rightarrow \Delta_A(a)$ and $b \in \mathcal{T}m(\Gamma.A, B)$, $\delta_1 : \Delta_1 \Rightarrow \Delta_B(b)$, which give rise to $(a, b) : A \times B$ and $E(\delta_0) \otimes \delta_1 : E(\Delta_0) \otimes \Delta_1 \Rightarrow E(\Delta_A(a)) \otimes \Delta_B(b)$. Similarly, we can justify the elimination rule. For the computation rule, the underlying intuitionistic terms will be equated by virtue of the CwF, and the associated morphisms are equal. $\square$

For our other positive type formers we can follow a similar approach, where we just need to assume that the model of the host

theory supports the underlying intuitionistic types (as spelled out, e.g., by Angiuli and Gratzer [8]).

LEMMA 3.2. An $\text{ICwF}(Cx, Sp)$ models $\top^\circ$, $\perp^\circ$ and $\mathbb{B}^\circ$ if Cx supports $\top$, $\perp$ and $\mathbb{B}$, respectively.

PROOF. Straightforward, where the dependent supply in each case is $\diamond$ since we do not consider these data types as resources. $\square$

Our linear W -types require a bit more care to model, but will also not require any more structure than is already present in any ICwF , and that the underlying CwF supports usual W -types [1, 40].

DEFINITION 3.5. A $\text{CwF } Cx$ supports $\mathcal{W}$ -types if for any $\Gamma : Cx$, $A \in \mathcal{T}y(\Gamma)$ and $B \in \mathcal{T}y(\Gamma.A)$, the polynomial endofunctor associated with A and B has an initial algebra. $\triangle$

Joining together a finite number of supplies with finsply just utilises the finite monoidal product. We thereby have everything at hand to model our linear W -types.

LEMMA 3.3. An $\text{ICwF}(Cx, Sp)$ models $\mathcal{W}^{*E}$ if Cx supports $\mathcal{W}$.

PROOF. Given linear types (A, Δ_A) and (B, Δ_B) , we interpret $\mathcal{W}^{*E}(A, B)$ using the initial algebra of the corresponding $\mathcal{W}$ -type and supply $\text{elim}_{\mathcal{W}}(x\Delta.(E(\Delta_A(x)) \otimes \bigotimes_{y:B(x)} \Delta(y)), q)$, using again term language to refer to the unique eliminator of $\mathcal{W}$. The introduction, elimination and computation rules follow in similar fashion to those in Lemma 3.1, using the eliminator of $\mathcal{W}$ judiciously when justifying the elimination rule for $\mathcal{W}^*$ (details can be found in the artefact described in Section 5). $\square$

The linear function type is the only type former that requires a bit more structure on our ICwF . First recall what it means for our host theory to support function types.

DEFINITION 3.6. A $\text{CwF } Cx$ supports dependent functions if for $\Gamma : Cx$ we have an operation $(-) \rightarrow (-) : (\prod_{A:\mathcal{T}y(\Gamma)} \mathcal{T}y(\Gamma.A)) \rightarrow \mathcal{T}y(\Gamma)$ and $\mathcal{T}m(\Gamma, A \rightarrow B) \cong \mathcal{T}m(\Gamma.A, B)$ all natural in Γ . $\triangle$

Additionally, we need exponential objects in our indexed symmetrical monoidal categories as well as an adjoint $\forall_A$ which allows us to turn a supply living in some context $\Gamma.A$ to one living in Γ . This allows us to bind a variable in a supply, and can be understood as an inverse principle to context extension for supplies. This quantifier is a relatively natural assumption—after all, we will need to bring together our linear calculus with the structural rules of the host theory somehow—and has in fact appeared in some form already in the literature [10, 51].

Similar to before, we again slightly generalise our considerations and consider function types whose domain is subjected to some given functor E .

LEMMA 3.4. An $\text{ICwF}(Cx, Sp)$ models $E\text{-}\multimap$ if each $Sp(\Gamma)$ is closed and $Sp(p_A) : Sp(\Gamma) \rightarrow Sp(\Gamma.A)$ has a right adjoint $\forall_A$ for any $\Gamma : Cx$ and $A \in \mathcal{T}y(\Gamma)$ natural in Γ .

PROOF. If all our supply categories are closed, our indexed category is in fact given by $Sp : Cx^{op} \rightarrow \text{SMCCat}$ and we write $[-, -]$ for the exponential of $Sp(\Gamma)$. The type formation rule is interpreted by taking two linear types (A, Δ_A) and (B, Δ_B) to the linear type $(A \rightarrow B, \forall_A[E(\Delta_A(q)), \Delta_B(\text{app}(q[p], q))])$, using app for the application principle derivable from Cx supporting function types. Intuitively, inhabitants of this linear type contain an intuitionistic

function f and an internalised morphism which witnesses that for any input $x : A$, $E(\Delta_A(x))$ can be turned into $\Delta_B(f\ x)$.

In the introduction rule, we are given $b \in \mathcal{T}m(\Gamma.A, B)$ and $\delta : \Delta \otimes E(\Delta_A(x)) \Rightarrow \Delta_B(b)$. The linear dependent function is then defined as $\lambda x.b \in \mathcal{T}m(\Gamma, A \rightarrow B)$, and the forwards direction of the isomorphism between $\text{hom}_{Sp(\Gamma.A)}(Sp(p_A)(\Delta_0), \Delta_1)$ and $\text{hom}_{Sp(\Gamma)}(\Delta_0, \forall_A(\Delta_1))$ as well as currying gives us the required morphism $\Delta \Rightarrow \forall_{x:A}[E(\Delta_A(x)), \Delta_B(b)]$. We model the application rule similarly using the inverse morphisms. Soundness of our interpretation follows immediately from soundness of our CwF and the associated supply morphisms being equal. $\square$

3.3 Models and Relation to Quantitative CwFs

We have seen that models of our syntax can be characterised as a rather straightforward combination of models of linear logic indexed in a model of DTT. This also simplifies our work when we want to evince some concrete models.

EXAMPLE 3.1. The *sets-and-relations* ICwF is given by an extension to the set model of type theory [8, Section 3.5] which works as follows.

- Cx is the largest Grothendieck universe $\mathcal{U}$,
- each $\Gamma : Cx$ is a set in $\mathcal{U}$, substitutions are functions between these sets,
- a type $A \in \mathcal{T}y(\Gamma)$ is a function $A : \Gamma \rightarrow \mathcal{U}$ and context extension $\Gamma.A$ as $\prod_{x \in \Gamma} A(x)$,
- a term $\mathcal{T}m(\Gamma, A)$ is an indexed products $\prod_{x \in \Gamma} A(x)$, and
- the intuitionistic type-formers are interpreted as their set-theoretic counterparts.

Since our functor Sp is embedded as a type into our theory, we also have that $Sp(\Gamma)$ gives rise to some type sply , which is interpreted as a function $\Gamma \rightarrow \mathcal{U}$, and each $\Delta : Sp(\Gamma)$ gives rise to some $\Delta \in \mathcal{T}m(\Gamma, \text{sply})$ which is interpreted as a product $\prod_{x \in \Gamma} \text{sply}(x)$. The embedded hom-sets are interpreted similarly. We can regard each set $\text{sply}(x)$ as a SMCC in the standard way, with cartesian product giving us both the monoidal structure and the internalised hom-sets. A ground term $x \in \Gamma$ is mapped by ι_Γ to the singleton supply.

To interpret function types, we define $\forall_A : Sp(\Gamma.A) \rightarrow Sp(\Gamma)$ by sending $\phi : \prod_{f \in \prod_{x \in \Gamma} A(x)} \text{sply}(f)$ to $(x \in \Gamma) \mapsto \prod_{a \in A(x)} \phi(x, a)$ (note that $Sp(\Gamma.A) = \prod_{x \in \Gamma} A(x) \rightarrow \mathcal{U}$). $\triangle$

This model gives us confidence in our system being aptly called a dependent linear type theory as it is just an indexed version of the usual sets-and-relations model of linear logic. As an application, we can use it to show that there really is no projection turning (x, y) into x as argued in Example 2.5 since $((x^m), y)$ will in general not be related with x , where x^m denotes an m -tuple of x 's.

We have motivated our setup with $\vdash$ being only relevant for specification, while everything derived in $\Vdash$ being the programs that we actually care about—akin to the 0- and 1-fragments of QTT [9]. To substantiate this point, we give a realisability model in which only $\Vdash$ is realised. This model is very similar to that of Atkey [9] for QTT, where our remit is simplified by the fact that we derived the quantitative aspects of our theory instead of annotating the structural rules of our theory with a resource algebra. We can hence just use standard Linear Combinatory Algebras [5] and do

not have to equip its elements with a modality taking into account the resource algebra.

DEFINITION 3.7. A BCI-algebra is given by a set $\mathcal{A}$, a binary operation $(-) \cdot (-)$ written left-associatively and elements $B, C, I \in \mathcal{A}$ such that $B \cdot x \cdot y \cdot z = x \cdot (y \cdot z)$, $C \cdot x \cdot y \cdot z = x \cdot z \cdot y$ and $I \cdot x = x$. $\triangle$

We do need the additional structure for modelling booleans introduced by Atkey [9] since boolean values can effect a program's behaviour, despite not being present in the supplies.

DEFINITION 3.8. A BCI-algebra $\mathcal{A}$ supports booleans if we have elements $T, F \in \mathcal{A}$ and a function $E : \mathcal{A} \times \mathcal{A} \rightarrow \mathcal{A}$ such that $E(p, q) \cdot T = p$ and $E(p, q) \cdot F = q$. $\triangle$

Any BCI-algebra $\mathcal{A}$ gives rise to a symmetric monoidal closed category of assemblies $\mathbf{Asm}(\mathcal{A})$ [33] which has objects pairs $(X, \models_X)$, where X is a set of extensional meanings and $\models_X : \mathcal{A} \times X$ captures when an element $a \in \mathcal{A}$ realises some $x \in X$, such that any element is realisable, i.e., for any $x \in X$ we have $a \models_X x$ for some $a \in \mathcal{A}$. A morphism between $(X, \models_X)$ and $(Y, \models_Y)$ is a realisable function $f : X \rightarrow Y$, i.e., there is an $a_f \in \mathcal{A}$ such that $a \models_X x$ implies $a_f \cdot a \models_Y f(x)$.

As pointed out by Atkey [10], the original realisability model of QTT [9] was faulty since also contexts which only had 0-ed variables had realisers, even though the 0-fragment of QTT ought to be erased. The dichotomy between contexts and supplies makes things easier for us since this we can fully erase the $\vdash$ fragment while realising the $\Vdash$ fragment of our syntax. Note that a supply with only 0-ed variables $(x : A) \wedge 0$ is definitionally equal to the empty supply.

EXAMPLE 3.2. For any BCI-algebra $\mathcal{A}$ which supports booleans, the realisability model is given by a CwF Cx of sets and functions which is indexed in sub-categories of $\mathbf{Asm}(\mathcal{A})$ as follows: a set Γ in Cx is sent by Sp to the category of assemblies $(\Delta, \models_\Delta)$ such that Δ is a subset of Γ . The elements of Γ can be understood as the ground terms, and ι_Γ sends each x to the singleton assembly containing x . The contexts hence capture what resources might be contemplated, while the supplies contain the resources actually available for computation. The model supports function types if for any $a \models_\Delta x$ at $(\Delta, \models_\Delta) : Sp(\Gamma, A)$ we have $(\Delta', \models_{\Delta'}) : Sp(\Gamma)$, a function $\forall : \Delta \rightarrow \Delta'$ and $a_x \in \mathcal{A}$ such that $a_x \cdot a \models_{\Delta'} \forall x$. $\triangle$

Our realisability model justifies that we only treat terms derived in $\Vdash$ as relevant for computation, and that the $\vdash$ fragment can be erased similarly to the 0-fragment of QTT. Note that we can also erase open programs, i.e., terms derived in a non-empty intuitionistic context Γ , provided that the supply does not mention any hypotheses, as has been worked out for QTT by Abel et al. [2].

The relationship with QTT goes deeper, and at least for a resource algebra that can be presented inductively our syntax is more general. We refer to Atkey [9] for a definition of Quantitative Categories with Families (QCwF).

LEMMA 3.5. An lCwF (Cx, Sp) is a $\mathbb{N}$ -QCwF if Cx supports a natural numbers type.

PROOF. For the CwF underlying the QCwF we have Cx , while we use the collection of all supplies for the category of contexts with resource annotations, i.e., $\mathcal{L} := \coprod_{\Gamma : Cx} Sp(\Gamma)$. The functor

$U : \mathcal{L} \rightarrow Cx$ is given by the first projection, and the addition and scaling structure can be derived similarly as in Section 2.3. The resourced terms of the QCwF are precisely our linear terms, and resourced context extension is given by extending supplies with an exponentiated variable $x \hat{=} m$ (the required natural transformations for resourced context extension hold strictly in our model). $\square$

4 Re-embedding Full Intuitionistic Logic

We have seen in Example 3.2 that when computing with our theory, we can erase the $\vdash$ fragment and only keep the $\Vdash$ fragment. The latter only allows value-linear terms, however—while this is a quite large class of terms due to our dynamic notion of multiplicity, we still cannot type all terms, e.g., we have no first projection function. To rectify this, we can use the standard approach of recovering full intuitionistic logic inside linear logic by equipping resources whose values might be used arbitrarily often with a modality “!”.

$$\frac{\Gamma \vdash \Delta \text{ sply}}{\Gamma \vdash !\Delta \text{ sply}} \text{!sply}$$

We also add structural rules which allow us to arbitrarily duplicate and discard supplies annotated with “!”, these are entirely standard [14] and we refer the interested reader to the artefact described in Section 5 for details.

In order to abstract over variables annotated with “!” we introduce a “!”-function type whose rules are analogous to the linear function type we gave in Section 2.2. The formation rule for such a type $!(x : A) \multimap B(x)$ applies to any two linear types A and B , and the introduction and elimination rules are as follows.

$$\frac{\Gamma \vdash \Delta \text{ sply} \quad \Delta, !(x : A) \Vdash b(x) : B(x)}{\Delta \Vdash \lambda !x. b : !(x : A) \multimap B(x)} \text{!}\multimap\text{I}$$

$$\frac{\Delta_0 \Vdash f : !(x : A) \multimap B(x) \quad \Delta_1 \Vdash a : A}{\Delta_0, !\Delta_1 \Vdash f a : B(a)} \text{!}\multimap\text{APP}$$

The computation and substitution rules for !-functions are standard, and by defining $(!(x : A) \multimap B(x))^\bullet := (x : A^\bullet) \multimap B(x)^\bullet$ we recover the underlying function type.

Equipping our linear type system with the modality “!” therefore gives us the full power of intuitionistic logic.

EXAMPLE 4.1. We can construct a function of type $!(A \otimes B) \multimap A$ in the $\Vdash$ fragment of our system extended with “!”.

Semantically, the rules for “!” corresponds to requiring an exponential comonad [14] in each $Sp(\Gamma)$. The semantic justification for function types we gave in Lemma 3.4 also applies to our “!”-functions using functoriality of “!”. We thereby have established that we have recovered intuitionistic logic in our linear fragment $\Vdash$.

LEMMA 4.1. For an lCwF (Cx, Sp) in which each $Sp(\Gamma)$ has an exponential comonad, $Sp(\Gamma)$ is cartesian closed.

5 Implementation of the Type System in Agda

The intuitionistic fragment of our theory is a standard DTT, which makes us hopeful that equipping existing dependently typed languages with our type system is relatively easy. Moreover, the semantics for our theory presented in Section 3 can be formalised in DTT, suggesting an implementation of our type system in which

linear terms are represented as terms of the DTT equipped with an inhabitant of the supply that we associated with each linear type in Section 3.2. In other words, by using a DTT both to construct programs, and as a meta-language to impose the semantics behind our linear type system, we can obtain a prototype for our type system. While it is not particularly practical—we have to construct the morphisms witnessing linearity by hand—we can guarantee value linearity of programs constructed in the linear fragment. We describe how to implement this prototype in Agda [43], the construction can be carried out similarly in other dependently typed languages such as Rocq [13] or Lean [23]. We have uploaded the artefact for the review process to the anonymous git-repo <https://github.com/anonforlics26/dlitt>.

To equip Agda with supplies, we introduce inductive types `sply` and `_!_`, where the latter is a relation on `sply`. The constructors of these data types correspond exactly to the objects and morphisms of our symmetric monoidal categories. Moreover, we equip `sply` with a constructor `!` which embeds any ground term as a supply.

Furthermore, we introduce a datatype `ltyp` whose constructors correspond to the formation rules of our linear types. We can then utilise the translation $(-)^{\bullet}$ (given inductively in Section 2) as well as the interpretations of our type formers (Lemmas 3.1–3.4) to compute from any linear type the underlying intuitionistic type A and the supply capturing how A is considered a resource.

$\llbracket _ \rrbracket : \text{ltyp } \ell \rightarrow \Sigma(A : \text{ityp } \ell) (A \rightarrow \text{sply } \ell)$

With this interpretation we obtain a function $_^{\bullet} : \text{ltyp} \rightarrow \text{ityp}$ and use Agda’s `syntax` feature to be able to write $a : A$ for the supply corresponding to linear type A at value a .

With this we have everything at hand to write and type-check terms in our system. For example, the variable rule is justified by the identity morphism.

$\text{Var}^{\circ} : (A : \text{ltyp } \ell) \rightarrow (x : A^{\bullet}) \rightarrow x : A \vdash x : A$
 $\text{Var}^{\circ} A x = \text{id } (x : A)$

We can derive the introduction, elimination and computation rules for all the type formers exactly as described in the proofs of Lemmas 3.1–3.4, the artefact hence provides a formalisation of these lemmas. Moreover, we can implement all the example programs that we gave, e.g., we can show admissible the recursor for binary trees, where t, n and l are in the context as terms of the respective underlying intuitionistic types.

$(\Delta_0 \vdash t : \text{BTree } A)$
 $\rightarrow (\{y z : C^{\bullet}\} \rightarrow y : C, z : C \vdash n y z : C)$
 $\rightarrow (\{x : A^{\bullet}\} \rightarrow \Delta_1, x : A \vdash l x : C)$
 $\rightarrow \Delta_0, \Delta_1 \wedge \text{leafs } t \vdash \text{recBTree } n l : C$

By equipping `sply` and `_!_` with a linear exponential comonad “!” as explained in Section 4, we can also derive a first projection function in the linear fragment with the appropriate type.

$\diamond \vdash \text{fst} : !(A \otimes B) \multimap A$

When we execute programs derived in `_!_`, we are only interested in the intuitionistic term underlying our linear judgments, which we can recover with an evaluation function that amounts to taking the first projection in the host theory.

$\text{eval} : \Delta \vdash x : A \rightarrow A^{\bullet}$

While our implementation does not erase the intuitionistic fragment, as would be justified by the realisability model (Example 3.2), we obtain a prototype of our type system in which a witness of $\Delta \vdash a : A$ guarantees that a is made up of the resources Δ .

6 Related Work

There is a large body of work on dependent linear type theories, we sketch the main strands and how they relate to our type system.

Quantitative and graded type theories. Our work has been greatly inspired by quantitative and graded type theories [2, 9, 39, 44], which also stratify their syntax into two levels, one supporting full intuitionistic type theory and the other one a resource-aware linear calculus (the 0-fragment and the 1-fragment, respectively, in QTT [9]). Our type system can be seen as taking McBride’s approach further: we use the underlying type theory not only to handle type dependencies, but also to compute resource annotations. This allows us to precisely specify resource usage with dynamic multiplicities, while the aforementioned systems only allow for static resource annotations where multiplicities are drawn from some resource algebra. While this algebra can be the natural numbers, in practice mostly a semiring containing 0, 1 and ω is used [18, 44], where ω allows for arbitrary usage of some variable. While we could in principle also use an inductive type to represent such a resource algebra, our usage of a modality “!” introduced in Section 4 is a more standard approach to recovering full intuitionistic logic in a linear system.

Inductive types in QTT have been characterised as Quantitative Polynomial Functors by Nakov and Nordvall Forsberg [42]. In contrast to their approach, we did not change the interpretation of W-types per se, but instead made them resource-aware by reflecting their computational behaviour in the supply that is associated with each linear type (and we did not require additional structures on linear Categories with Families). Our semantics justifies the general elimination rule for W-types ($W^{\circ}E$), which subsumes all elimination rules that have been considered in the literature so far [34, 39]. Further work is necessary to understand how our interpretation of linear W-types relates to Quantitative Polynomial Functors.

Dialectica. Our system has also been influenced by the Dialectica translation for type theory proposed by Pédröt [45, 46]. Gödel’s Dialectica translation [54] provides a very general recipe to turn an intuitionistic calculus into a linear one [24, 25, 41], and our system can be seen as turning the target of the Dialectica translation into a fully fledged type theory by equipping the indexed categories with more structure to support function types and a linear exponential comonad.

Recently, Doré [26] has given a programming technique which allows for deriving linear judgments in an intuitionistic type theory, using an approach very similar to ours. They use finite multisets encoded as higher inductive types [50] to represent supplies in Cubical Agda [53]. In contrast to our system, theirs does not support function types. Moreover, they embed *all* terms into supplies using a natural transformation ι from intuitionistic terms to supplies, whereas we only embed ground terms and then interpret type formers purposefully using dependent eliminators. This saves us from having to require additional properties of ι , e.g., that it

is strongly monoidal with respect to pairs of terms, significantly simplifying our model. The treatment of inductive types by Doré [26] is also quite ad-hoc, whereas we give a general treatment of linear W -types.

Index terms in dℓPCF. The idea of non-static resource annotations has been used fruitfully by Dal Lago and Gaboardi [21] to give a highly expressive type system for PCF, called dℓPCF. Their theory works, similarly to ours, by embedding a model of linear logic—in their case history-free game semantics [6]—in an intuitionistic calculus. The typing judgment of dℓPCF comes equipped with an *index term* which specifies the cost of a program. Dal Lago and Gaboardi are primarily interested in measuring the runtime of functional programs, which means that cost is defined differently than in our setting, e.g., they consider the cost of doubling a natural number n presented in unary as n since it requires n recursive calls, whereas our system treats any function between the natural numbers as not manipulating any resources. These differences are not crucial however, and we could also make our notion of cost more intensional by introducing a resource-relevant diamond type as proposed by Hofmann [32] and Atkey [10]. The calculus of Dal Lago and Gaboardi [21] does not allow for internalising the cost-annotated judgments as some sort of linear function type, which means dℓPCF is limited to a more global analysis of cost.

Dependent linear type theories. Our approach differs from prior work on dependent linear type theory as we embed linear logic into type theory, as opposed to combining two logics as equals. Many approaches [20, 35, 36, 51] are based on linear-non-linear logic [12, 15], distinguishing between intuitionistic and linear variables and only allowing dependencies on intuitionistic variables. While we also differentiate between intuitionistic and linear types in our theory, this is not a dichotomy—rather, the linear types are more nuanced, substructural versions of the intuitionistic types of the host theory. In particular, we allow dependencies on variables which are treated as resources and hence have a more expressive system.

7 Conclusions

Our system presents a novel way to combine intuitionistic DTT with linear logic. We can give precise resource annotations to a large class of programs, in particular to many higher-order, branching and recursive programs. Our system presents both a generalisation and simplification of previous type systems [9, 19, 35, 39, 51] since we can adequately type a larger class of programs, while reusing well-studied structures and languages to provide semantics and an implementation for our system.

In the future we hope to broaden the scope of our approach, and to utilise it for different applications of linear logic.

Embedding other substructural logics apart from linear logic can be done straightforwardly by stipulating the relevant structural rules; we also want to explore if we can embed classical linear logic in order to obtain a classical linear logic with predicates. So far, we have only added inductive types to our system, but we expect that we can also incorporate coinductive types and dependent W -types [28] in our linear system, extending the work of Doré [26], who considers multiplicities drawn from the conatural numbers. More work is necessary to understand if and how our approach can be

used to linearise univalent type theories [17, 50], possibly giving rise to a linear homotopy type theory [48].

Our system captures what we called *value linearity*, which is only one of many applications of linear logic. We also want to study if our approach can be used for complexity theory [11, 22, 30, 32], akin to how Atkey [10] has used QTT to characterise certain complexity classes. Similarly, Atkey [9] has suggested that making booleans resource-aware gives rise to a system which captures reversible computations [3], which points to applications of our system for quantum computing. Lastly, we can view classical linear propositions as *sessions*, giving rise to a calculus for concurrent computation [19, 49]. We hope that our approach will prove versatile and be useful in these applications, only requiring small changes to the interpretation of linear types to obtain languages tailored to different notions of linearity.

References

- [1] Michael Abbott, Thorsten Altenkirch, and Neil Ghani. 2005. Containers: Constructing Strictly Positive Types. *Theoretical Computer Science* 342, 1 (2005), 3–27. <https://doi.org/10.1016/j.tcs.2005.06.002> Applied Semantics: Selected Topics.
- [2] Andreas Abel, Nils Anders Danielsson, and Oskar Eriksson. 2023. A Graded Modal Dependent Type Theory With a Universe and Erasure, Formalized. *Proc. ACM Program. Lang.* 7, ICFP, Article 220 (Aug. 2023), 35 pages. <https://doi.org/10.1145/3607862>
- [3] Samson Abramsky. 2005. A Structural Approach To Reversible Computation. *Theoretical Computer Science* 347, 3 (2005), 441–464. <https://doi.org/10.1016/j.tcs.2005.07.002>
- [4] Samson Abramsky and Ross Duncan. 2006. A Categorical Quantum Logic. *Mathematical Structures in Computer Science* 16, 3 (2006), 469–489.
- [5] Samson Abramsky, Esfandiar Haghighverdi, and Philip Scott. 2002. Geometry of Interaction and Linear Combinatory Algebras. *Mathematical Structures in Computer Science* 12, 5 (2002), 625–665.
- [6] Samson Abramsky, Radha Jagadeesan, and Pasquale Malacaria. 2000. Full Abstraction for Pcf. *Information and Computation* 163, 2 (2000), 409–470. <https://doi.org/10.1006/inco.2000.2930>
- [7] Peter Aczel. 1978. The Type Theoretic Interpretation of Constructive Set Theory. In *Logic Colloquium '77*, Angus Macintyre, Leszek Pacholski, and Jeff Paris (Eds.). Studies in Logic and the Foundations of Mathematics, Vol. 96. Elsevier, 55–66. [https://doi.org/10.1016/S0049-237X\(08\)71989-X](https://doi.org/10.1016/S0049-237X(08)71989-X)
- [8] Carlo Angiuli and Daniel Gratzer. 2025. *Principles of Dependent Type Theory*. in preparation. <https://www.danielgratzer.com/papers/type-theory-book.pdf>
- [9] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science LICS* (2018), 56–65. <https://doi.org/10.1145/3209108.3209189>
- [10] Robert Atkey. 2024. Polynomial Time and Dependent Types. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 2288–2317.
- [11] Patrick Baillot, Marco Gaboardi, and Virgile Mogbil. 2010. A polytime functional language from light linear logic. In *Proceedings of the 19th European Conference on Programming Languages and Systems (Paphos, Cyprus) (ESOP'10)*. Springer-Verlag, Berlin, Heidelberg, 104–124. https://doi.org/10.1007/978-3-642-11957-6_7
- [12] Andrew Barber and Gordon Plotkin. 1996. *Dual intuitionistic linear logic*. Technical Report ECS-LFCS-96-347.
- [13] Bruno Barras, Samuel Boutin, Cristina Cornes, Judicaël Courant, Jean-Christophe Filliâtre, Eduardo Giménez, Hugo Herbelin, Gérard Huet, César Muñoz, Chetan Murthy, Catherine Parent, Christine Paulin-Mohring, Amokrane Saïbi, and Benjamin Werner. 1997. *The Coq Proof Assistant Reference Manual : Version 6.1*. Research Report RT-0203. INRIA. 214 pages. <https://hal.inria.fr/inria-00069968>
- [14] Nick Benton, Gavin Bierman, Valeria de Paiva, and Martin Hyland. 1993. A term calculus for Intuitionistic Linear Logic. In *Typed Lambda Calculi and Applications*, Marc Bezem and Jan Friso Groote (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 75–90.
- [15] P. N. Benton. 1995. A mixed linear and non-linear logic: Proofs, terms and models. In *Computer Science Logic*, Leszek Pacholski and Jerzy Tiuryn (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 121–135.
- [16] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2017. Linear Haskell: practical linearity in a higher-order polymorphic language. *Proc. ACM Program. Lang.* 2, POPL, Article 5 (Dec. 2017), 29 pages. <https://doi.org/10.1145/3158093>
- [17] Marc Bezem, Thierry Coquand, and Simon Huber. 2014. A Model of Type Theory in Cubical Sets. In *19th International Conference on Types for Proofs and Programs (TYPES) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 26)*, Ralph

- Matthes and Aleksy Schubert (Eds.), 107–128. <https://doi.org/10.4230/LIPIcs.TYPES.2013.107>
- [18] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 194)*, Anders Möller and Manu Sridharan (Eds.), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:26. <https://doi.org/10.4230/LIPIcs.ECOOP.2021.9>
- [19] Luis Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR 2010 - Concurrency Theory*, Paul Gastin and François Laroussinie (Eds.), Springer Berlin Heidelberg, Berlin, Heidelberg, 222–236. https://doi.org/10.1007/978-3-642-15375-4_16
- [20] Ilario Cervesato and Frank Pfenning. 2002. A linear logical framework. *Information and Computation* 179, 1 (Nov. 2002), 19–75. <https://doi.org/10.1006/inco.2001.2951>
- [21] Ugo Dal Lago and Marco Gaboardi. 2011. Linear Dependent Types and Relative Completeness. *Proceedings of the 26th Annual ACM/IEEE Symposium on Logic in Computer Science LICS* (2011), 133–142. <https://doi.org/10.1109/LICS.2011.22>
- [22] Ugo Dal Lago and Martin Hofmann. 2011. Realizability Models and Implicit Complexity. *Theoretical Computer Science* 412, 20 (2011), 2029–2047. <https://doi.org/10.1016/j.tcs.2010.12.025> Girard’s Festschrift.
- [23] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. 2015. The Lean theorem prover (system description). *CADE* (2015), 378–388. https://doi.org/10.1007/978-3-319-21401-6_26
- [24] Valeria Correa Vaz de Paiva. 1990. *The Dialectica Categories*. Ph.D. Dissertation. University of Cambridge, UK.
- [25] Valeria Correa Vaz de Paiva. 1991. *The Dialectica categories*. Technical Report UCAM-CL-TR-213. University of Cambridge, Computer Laboratory. <https://doi.org/10.48456/tr-213>
- [26] Maximilian Doré. 2025. Linear Types With Dynamic Multiplicities in Dependent Type Theory (Functional Pearl). *Proc. ACM Program. Lang.* 9, ICFP, Article 262 (2025), 21 pages. <https://doi.org/10.1145/3747531>
- [27] Peter Dybjer. 1996. Internal type theory. In *Types for Proofs and Programs (TYPES) 1995*, Stefano Berardi and Mario Coppo (Eds.), Springer, 120–134. https://doi.org/10.1007/3-540-61780-9_66
- [28] Nicola Gambino and Martin Hyland. 2004. Wellfounded Trees and Dependent Polynomial Functors. In *Types for Proofs and Programs*, Stefano Berardi, Mario Coppo, and Ferruccio Damiani (Eds.), Springer Berlin Heidelberg, Berlin, Heidelberg, 210–225. https://doi.org/10.1007/978-3-540-24849-1_14
- [29] Jean-Yves Girard. 1987. Linear Logic. *Theoretical computer science* 50, 1 (1987), 1–101.
- [30] Jean-Yves Girard. 1994. Light linear logic. In *International Workshop on Logic and Computational Complexity*. Springer, 145–176.
- [31] Martin Hofmann. 1997. *Syntax and semantics of dependent types*. Springer London, Chapter 2, 13–54. https://doi.org/10.1007/978-1-4471-0963-1_2
- [32] Martin Hofmann. 1999. Linear Types and Non Size-Increasing Polynomial Time Computation. In *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science (LICS ’99)*. IEEE Computer Society, USA, 464.
- [33] Naohiko Hoshino. 2007. Linear realizability. In *Proceedings of the 21st International Conference, and Proceedings of the 16th Annual Conference on Computer Science Logic (Lausanne, Switzerland) (CSL’07/EACSL’07)*. Springer-Verlag, Berlin, Heidelberg, 420–434.
- [34] Yulong Huang and Jeremy Yallop. 2025. Towards Quantitative Inductive Families. *TYPES 2025* (2025).
- [35] Neelakantan R. Krishnaswami, Pierre Pradic, and Nick Benton. 2015. Integrating Linear and Dependent Types. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Mumbai, India) (POPL ’15)*. Association for Computing Machinery, New York, NY, USA, 17–30. <https://doi.org/10.1145/2676726.2676969>
- [36] Daniel R. Licata, Michael Shulman, and Mitchell Riley. 2017. A Fibrational Framework for Substructural and Modal Logics. In *2nd International Conference on Formal Structures for Computation and Deduction (FSCD 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 84)*, Dale Miller (Ed.), Dagstuhl, Germany, 25:1–25:22. <https://doi.org/10.4230/LIPIcs.FSCD.2017.25>
- [37] Per Martin-Löf. 1982. Constructive Mathematics and Computer Programming. *Studies in Logic and the Foundations of Mathematics* 104 (1982), 153 – 175. <https://dl.acm.org/doi/10.5555/3721.3731>
- [38] Per Martin-Löf. 1984. *Intuitionistic type theory*. Bibliopolis.
- [39] Conor McBride. 2016. I Got Plenty o’ Nuttin’. In *A List of Successes That Can Change the World: Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday*, Sam Lindley, Conor McBride, Phil Trinder, and Don Sannella (Eds.), Springer, 207–233. https://doi.org/10.1007/978-3-319-30936-1_12
- [40] Ieke Moerdijk and Erik Palmgren. 2000. Wellfounded Trees in Categories. *Annals of Pure and Applied Logic* 104, 1 (2000), 189 – 218. [https://doi.org/10.1016/S0168-0072\(00\)00012-9](https://doi.org/10.1016/S0168-0072(00)00012-9)
- [41] Sean K. Moss and Tamara von Glehn. 2018. Dialectica models of type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (Oxford, United Kingdom) (LICS ’18)*. Association for Computing Machinery, New York, NY, USA, 739–748. <https://doi.org/10.1145/3209108.3209207>
- [42] Georgi Nakov and Fredrik Nordvall Forsberg. 2022. Quantitative Polynomial Functors. In *27th International Conference on Types for Proofs and Programs (TYPES) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 239)*, Henning Basold, Jesper Cockx, and Silvia Ghilezan (Eds.), Dagstuhl, Germany, 10:1–10:22. <https://doi.org/10.4230/LIPIcs.TYPES.2021.10>
- [43] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph.D. Dissertation. Chalmers University of Technology and Göteborg University.
- [44] Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. 2019. Quantitative program reasoning with graded modal types. *Proc. ACM Program. Lang.* 3, ICFP, Article 110 (July 2019), 30 pages. <https://doi.org/10.1145/3341714>
- [45] Pierre-Marie Pédot. 2014. A functional functional interpretation. *Proceedings of the Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science LICS/CSL* (2014). <https://doi.org/10.1145/2603088.2603094>
- [46] Pierre-Marie Pédot. 2024. Dialectica the Ultimate. (2024). <https://www.p%C3%A9drot.fr/slides/tlla-07-24.pdf> Talk at Trends in Linear Logic and Applications.
- [47] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. 2021. Perceus: garbage free reference counting with reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (Virtual, Canada) (PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 96–111. <https://doi.org/10.1145/3453483.3454032>
- [48] Mitchell Riley. 2022. *A Bunched Homotopy Type Theory for Synthetic Stable Homotopy Theory*. Ph.D. Dissertation. Wesleyan University.
- [49] Bernardo Toninho, Luis Caires, and Frank Pfenning. 2011. Dependent session types via intuitionistic linear type theory. In *Proceedings of the 13th International ACM SIGPLAN Symposium on Principles and Practices of Declarative Programming (Odense, Denmark) (PPDP ’11)*. Association for Computing Machinery, New York, NY, USA, 161–172. <https://doi.org/10.1145/2003476.2003499>
- [50] The Univalent Foundations Program. 2013. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Available at <https://homotopytypetheory.org/book>, Institute for Advanced Study.
- [51] Matthijs Vákár. 2015. A Categorical Semantics for Linear Logical Frameworks. In *Foundations of Software Science and Computation Structures*, Andrew Pitts (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 102–116. https://doi.org/10.1007/978-3-662-46678-0_7
- [52] Matthijs Vákár. 2017. *In Search of Effectful Dependent Types*. Ph.D. Dissertation. University of Oxford.
- [53] Andrea Vezzosi, Anders Mörtberg, and Andreas Abel. 2021. Cubical Agda: a Dependently Typed Programming Language With Univalence and Higher Inductive Types. *Journal of Functional Programming* 31 (2021), e8.
- [54] Kurt Von Gödel. 1958. Über Eine Bisher Noch Nicht benützte Erweiterung Des Finiten Standpunktes. *Dialectica* 12, 3-4 (1958), 280–287. <https://doi.org/10.1111/j.1746-8361.1958.tb01464.x>

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009